

PRODUCT SYSTEMS / CONCEPTUAL ARCHITECTURE

HeyClicky Architecture Overview

A detailed conceptual guide to the conversational, screen-aware, and background work system

01

Realtime

Conversation and routing

02

Screen-aware

Situated answers and guidance

03

Background

Tools, actions, and artifacts

About This Document

This document explains the product at a conceptual level. It describes responsibilities, routing, tools, integrations, lifecycle, privacy, and boundaries without exposing hidden instructions, security controls, credentials, or implementation-sensitive policy text.

READING NOTE

This is an architectural explanation for internal discussion, onboarding, and alignment. It is not a security design, API reference, or promise that every implementation detail remains fixed.

Contents

01	Executive Summary
02	System at a Glance
03	Layer One: Realtime Conversation
04	Layer Two: Screen-Aware Answering
05	Layer Three: Background Agents
06	Conceptual Request Routing
07	Tool Categories
08	Connected Integrations and Composio
09	Background Agent Lifecycle
10	Artifacts and Handoff
11	Privacy and Data Handling
12	Limitations and Boundaries
13	Illustrative Scenarios
14	Operating Principles
15	Glossary

01

Executive Summary

HeyClicky is best understood as a coordinated system rather than a single assistant doing everything in one place.

The product separates fast conversation, screen-aware guidance, and longer-running work into distinct paths. This separation keeps ordinary dialogue responsive while allowing more capable tools and agents to be introduced only when the request genuinely needs them.

At the front is the realtime conversation layer. It maintains the live exchange, interprets intent, answers straightforward questions, and decides whether the request should stay conversational or be routed elsewhere. A screen-aware answer path handles questions that benefit from visual context, deeper reasoning, or current information. A background agent handles work that takes multiple steps, creates files, browses across sources, or acts through connected services.

Composio sits inside the connected-integration portion of the background workflow. It is not the foundation of every response. It becomes relevant when an approved task needs to read from or act within an external service such as Gmail or another connected application.

Core takeaways

- Fast dialogue stays on the fastest appropriate path.
- Screen context is used when the visible interface materially improves the answer.
- Background agents are reserved for work, not ordinary conversation.
- Connected integrations are invoked selectively and only for the requested workflow.
- Artifacts provide a durable result after the background task ends.

02

System at a Glance

The architecture follows a simple principle: use the lightest capable path, then escalate only when the task demands it.

A request begins in the conversational surface. The system identifies what the user is trying to accomplish, what context is available, and whether the request is answer-only or requires actual work. It then selects one of three primary execution paths.

The three paths cooperate, but they have different success criteria. The realtime layer succeeds when the exchange feels immediate and coherent. The screen-aware path succeeds when it provides a better answer because it can understand what is visible. The background path succeeds when it completes a multi-step outcome and returns evidence of that work, usually as a file, a created object, or a clear report.

Three coordinated paths

Realtime Conversation	Listen, interpret, respond, route	Immediate
Screen-Aware Answer	Use visible context, reason, guide	Seconds
Background Agent	Plan, use tools, create or act	Longer running

KEY IDEA

The layers are not a hierarchy of better and worse. Each is optimized for a different class of request.

03

Layer One: Realtime Conversation

This is the live conversational layer the user experiences directly.

Its first responsibility is continuity. It keeps track of the immediate exchange, resolves references such as 'that document' or 'open it', and maintains a natural conversational rhythm. It should avoid turning every small question into a heavy workflow.

Its second responsibility is triage. The layer distinguishes between an answer that can be produced immediately, a question that would be improved by screen or current-information context, and a request that requires a background worker. This routing decision is conceptual rather than a visible menu the user must operate.

Its third responsibility is coordination. When another path is used, the conversation layer explains what is happening in plain language, provides brief progress updates when appropriate, and presents the finished result without forcing the user to understand the underlying toolchain.

Typical responsibilities

- Live dialogue and turn-taking
- Intent recognition and reference resolution
- Simple factual or explanatory answers
- Lightweight local actions that can complete immediately
- Routing and user-facing status communication

What it deliberately avoids

- Launching a background task for every request
- Using connected integrations when no external service is needed
- Claiming that work is complete before an artifact or action is verified
- Overloading the user with internal implementation details

04

Layer Two: Screen-Aware Answering

This path is used when the answer becomes materially better by understanding the current screen or by applying a stronger answer process.

Screen awareness changes the interaction from generic advice to situated guidance. Instead of saying where a setting is usually found, the system can reason from the interface that is actually visible and explain what the user should look for next.

The path can also be used for harder questions, synthesis, drafting, or current-information lookups that do not require a long-running background workflow. The important distinction is that it primarily produces an answer or guidance, rather than executing a broad sequence of work across many tools.

Model and provider choices are intentionally abstracted in this document because they can change over time. The stable architectural concept is a specialized answer path with access to richer context than the realtime conversational loop.

Good fits

- Explaining a confusing interface currently on screen
- Pointing out a visible control or next step
- Summarizing or interpreting visible content
- Drafting text for a field the user is viewing
- Answering a difficult or current question without a long workflow

Boundary

- Screen awareness does not automatically authorize clicking, submitting, purchasing, sending, deleting, or changing an account. Answering and acting remain distinct responsibilities.

05

Layer Three: Background Agents

Background agents perform the requests that are better treated as jobs than as chat responses.

A background agent receives a bounded task, forms a working plan, uses the tools available to it, monitors its own progress, and produces a final outcome. The agent can continue while the live conversation remains available, which prevents a long operation from blocking the conversational experience.

Typical background work includes research across multiple sources, creation of Word or PDF documents, spreadsheet analysis, code changes, structured browsing, and connected-application workflows. The agent is expected to verify the result before reporting completion.

The output should be durable and inspectable. For a document task, that means a real file in a known location. For an integration task, it may mean a draft, event, issue, or updated record. For a research task, it may mean a report with sources and a concise summary.

A strong background task has

- A clear objective and finish condition
- A defined working location or destination
- The minimum tools needed for the job
- Progress milestones for longer operations
- Verification before the completion claim
- A clean handoff back to the user

Why background execution matters

- It separates conversational latency from job duration and gives multi-step work enough room to be done carefully rather than compressed into a single answer.

06

Conceptual Request Routing

Routing is the decision about whether the user needs an answer, situated guidance, or completed work.

The first routing question is whether the request can be satisfied with a direct response. If yes, the system answers immediately. If the answer depends on what is visible or on richer reasoning, the screen-aware answer path is selected. If the request requires a sequence of actions, file creation, prolonged browsing, or interaction with connected services, a background agent is appropriate.

Routing can change as the request becomes clearer. A user may begin with 'what am I looking at?' and then continue with 'turn this into a document'. The first request is screen-aware answering; the second is artifact-producing background work. The architecture supports that escalation without pretending the two requests are the same kind of operation.

Routing outcomes

Direct answer	Can be resolved conversationally	Explanation, short draft, simple guidance
Screen-aware answer	Visible context or deeper/current reasoning helps	Pointing, interpretation, situated next steps
Background agent	Multiple steps, tools, files, or external apps	Document, research report, code change, app workflow

Questions used to choose a path

- Is the request answer-only, or does it ask the system to do work?
- Would the current screen materially improve accuracy?
- Does the task require current external information?
- Does it involve multiple dependent steps?
- Will it create, modify, send, purchase, delete, or publish something?
- Is a connected service required, and is it already authorized?

07

Tool Categories

Tools extend what each path can observe or accomplish, but tools are selected by task rather than used by default.

A tool is a capability boundary. It may read a file, inspect the screen, browse the web, operate a local application, create an artifact, modify a codebase, or communicate with a connected service. The routing layer decides which class of capability is necessary; the agent then uses the narrowest suitable tool.

Conversation and reasoning	Interpretation, drafting, synthesis, planning	Answers and plans
Screen and computer use	Inspect visible UI, point, click, type, navigate	Guided or executed interface steps
Web and research	Current facts, source gathering, comparison	Verified current information
Local files and artifacts	Read, create, convert, render, validate	DOCX, PDF, CSV, HTML, images
Code and terminal	Inspect repositories, edit files, run checks	Working software changes
Connected integrations	Use authorized third-party services	Email drafts, calendar events, records

SELECTION PRINCIPLE

The presence of a tool does not mean it should be used. The system should prefer the least invasive capability that can reliably satisfy the request.

08

Connected Integrations and Composio

Composio provides an integration layer for approved background workflows involving external applications.

When a task requires a service such as Gmail, the background agent first identifies the appropriate integration capability and confirms that an active connection is available. It then uses a structured tool interface rather than handling account credentials directly.

This makes Composio a bridge between the agent and connected software. The bridge exposes actions such as searching messages, creating drafts, or reading application data in a structured form. Which actions are available depends on the connected application and the permissions granted to that connection.

Composio is selective infrastructure, not a universal transport for HeyClicky. A screen explanation, a local document, a code edit, or an ordinary answer may not involve Composio at all. It enters the picture only when the requested workflow needs a supported connected service.

Conceptual flow

- Identify that an external service is genuinely required
- Discover the correct integration action and its required inputs
- Confirm an active user-authorized connection
- Execute only the bounded action needed for the workflow
- Return a human-readable result and, where available, a link or created object

Important distinction

- The integration layer can provide access to an application, but it does not replace task planning, user intent, safety checks, or verification. Those remain responsibilities of the agent workflow.

09

Background Agent Lifecycle

A useful way to understand the background system is as a six-stage job lifecycle.

The lifecycle begins when the conversational layer turns a user request into a bounded task. The background agent then establishes context, plans the work, executes the necessary steps, validates the outcome, and returns the result to the conversational surface.

Six stages

- 1. Start:** Receive the objective, constraints, context, files, and intended destination.
- 2. Scope:** Resolve the project or working location and identify the finish condition.
- 3. Plan:** Break the job into ordered, verifiable steps and select the necessary tools.
- 4. Execute:** Carry out the work, using integrations only where the plan requires them.
- 5. Monitor and verify:** Track milestones, handle recoverable issues, inspect outputs, and confirm success.
- 6. Hand off:** Return a concise summary plus artifacts, created objects, or a clear blocker.

COMPLETION RULE

A background agent should report an outcome, not merely activity. 'Started creating the document' is progress; a verified DOCX and PDF are completion.

10

Artifacts and Handoff

Artifacts are the durable bridge between background work and the user.

An artifact may be a Word document, PDF, spreadsheet, webpage, image, code patch, or another file intended for review or sharing. Artifacts should be stored in a predictable project location, named clearly, and verified to exist before they are presented.

The handoff should answer three questions immediately: what was produced, where it is, and what it contains. A good handoff is short enough to scan but specific enough that the user can trust the result and continue working without searching through folders.

When several formats are requested, the primary shareable format should be listed first. Intermediate scripts, caches, render previews, and other working files should not be presented as user-facing deliverables.

Artifact quality checks

- The file exists and has nonzero size
- The intended sections and content are present
- The visual layout has been rendered and inspected
- Page breaks, headings, tables, and fonts are readable
- The final filenames are stable and descriptive
- Only user-facing deliverables are surfaced

11

Privacy and Data Handling

The high-level privacy principle is purpose limitation: use the context and access needed for the requested task, and avoid unrelated data.

Different requests expose different kinds of context. A normal conversation may need only the current exchange. Screen-aware help may use the current desktop view. A document task may use attached files. A connected workflow may access data from a service the user has linked. These contexts should not be treated as interchangeable or automatically combined.

External connections should be user-authorized and used for the specific workflow being performed. Credentials should remain within the connection provider or operating environment rather than being reproduced in an artifact or conversational response.

Generated documents should avoid embedding secrets, tokens, hidden instructions, or unnecessary personal data. If sensitive information is required for the task, the agent should minimize how much is copied and ensure the final deliverable matches the user's intended audience.

Purpose limitation	Use data only to complete the requested outcome.
Context separation	Do not assume screen, files, and connected apps are all needed together.
Least necessary access	Prefer the narrowest tool and smallest data scope that works.
User authorization	Treat connected services and consequential actions as permissioned capabilities.
Safe artifacts	Exclude credentials, hidden instructions, and irrelevant personal information.
Transparent handoff	State what was created or changed without overstating certainty.

Limitations and Boundaries

A clear architecture includes what the system does not do automatically.

Answering is not the same as acting. The system may explain a button without clicking it, draft an email without sending it, or describe a purchase flow without completing a purchase. Consequential actions require the appropriate execution path and may require confirmation.

Screen awareness is contextual, not omniscient. A screenshot may be incomplete, stale, obscured, or ambiguous. The system should ground guidance in what is actually visible and avoid inventing hidden state.

Background agents are capable but bounded by available tools, permissions, network access, file access, and the quality of the task definition. If a necessary capability is unavailable, the correct result is a clear blocker rather than a fabricated completion.

Integrations are not universal. A connected application must be supported, connected, and authorized, and the required action must be exposed by the integration. Not every HeyClicky interaction touches an integration provider.

Answer vs work	Advice or drafting does not imply execution
Visible vs hidden state	Screen context cannot prove what is not shown
Connected vs authorized	An available integration still depends on user-granted access
Progress vs completion	Starting a job is not the same as producing the result
Capability vs appropriateness	A tool should not be used merely because it exists

13

Illustrative Scenarios

These scenarios show how the same user relationship can move naturally between architecture paths.

Explain the page I am viewing	Screen-aware answer	Inspect the current screen, identify the relevant interface, explain what it means, and suggest the next step.
Turn that explanation into a detailed report	Background agent	Recover the conversation context, create a structured document, render it, verify it, and return Word and PDF artifacts.
Do I have any unread email about the project?	Background agent plus integration	Use an authorized Gmail connection to search the mailbox, summarize matching results, and avoid unrelated messages.
Help me reply to this message	Direct or screen-aware answer	Draft text from the visible or supplied message. Sending remains a separate action.
Update the website in my project	Background agent plus local tools	Locate the correct project, inspect the codebase, edit the files, run checks, and return the changed artifact or preview.

PATTERN

The route follows the requested outcome. Similar topics can use different paths when the user moves from understanding, to drafting, to acting.

Operating Principles

The architecture can be summarized as a set of practical product principles.

Conversation first	Keep ordinary interaction fast, natural, and coherent.
Escalate by need	Move to richer context or background work only when the request benefits.
Separate answers from actions	Make clear whether the system is advising, drafting, or executing.
Use the narrowest capable tool	Reduce unnecessary access, complexity, and side effects.
Treat integrations as selective	Use connected applications only inside workflows that require them.
Verify before claiming completion	Inspect outputs and confirm that the requested result exists.
Return durable outcomes	Prefer files, created objects, and clear summaries for substantial work.
Preserve user control	Keep consequential actions bounded, understandable, and permission-aware.

Glossary

Short definitions for the terms used throughout this overview.

Agent. A task-oriented process that can plan, use tools, verify results, and return an outcome.

Artifact. A durable user-facing output such as a document, PDF, spreadsheet, webpage, image, or code change.

Background agent. An agent launched for work that takes multiple steps or longer than a normal conversational response.

Composio. An integration layer that exposes structured actions for user-connected external applications.

Connected integration. An authorized bridge to an external service such as email, calendar, storage, or another application.

Context. Information available to a path, including conversation, screen state, files, project data, or connected application data.

Computer use. Tools that inspect and operate graphical interfaces through actions such as pointing, clicking, typing, and navigation.

Realtime layer. The live conversational path responsible for continuity, immediate responses, and routing.

Routing. The selection of the most appropriate path for a request: direct, screen-aware, or background work.

Screen-aware answer. An answer informed by the interface or content currently visible to the user.

Tool. A bounded capability used to observe information or perform an operation.

Verification. The checks performed before claiming that a requested action or artifact is complete.

Closing Perspective

HeyClicky's architecture is designed around a simple product promise: remain easy to talk to while still being able to do substantial work. The realtime layer protects responsiveness, the screen-aware path adds situated intelligence, and the background agent provides execution depth. Tools and integrations support those paths without becoming the product experience themselves. The user should encounter one coherent assistant, with the internal system selecting the right level of capability for each request.

IN ONE SENTENCE

HeyClicky routes each request to the lightest capable path, uses tools and integrations only when needed, and returns substantial work as a verified outcome.